

UNISENTIMENT

Sentiment Intelligence System

TECHNICAL REPORT

Architecture, NLP Pipeline & Implementation Reference

Group 12

Intro to Intelligence Systems

Dr. Patrick Gyaase

Group 12 — Sentiment Analysis System

Table of Contents

<u>1. System Architecture</u>	4
<u>1.1 Architectural Overview</u>	4
<u>1.2 Module Responsibilities</u>	4
<u>1.3 Request Flow</u>	4
<u>1.4 Frontend Stack</u>	4
<u>1.5 Why No Database</u>	5
<u>2. NLP Pipeline Deep Dive</u>	6
<u>2.1 Pipeline Stages</u>	6
<u>2.2 Negation Handling</u>	6
<u>2.3 Sarcasm Detection</u>	6
<u>2.4 Label Thresholds</u>	7
<u>2.5 Keyword Evidence Extraction</u>	7
<u>3. Aspect Detection & Scoring</u>	8
<u>3.1 Aspect Dictionary</u>	8
<u>3.2 Sentence-Level Aspect Isolation</u>	8
<u>3.3 Aspect Aggregation</u>	8
<u>4. Data Aggregation & Insights Engine</u>	9
<u>4.1 Insight Categories</u>	9
<u>4.2 Alert Generation</u>	9
<u>4.3 14-Day Trend Simulation</u>	9
<u>4.4 Keyword Frequency Analysis</u>	10
<u>5. API Reference</u>	11
<u>5.1 GET /</u>	11
<u>5.2 POST /analyse</u>	11
<u>5.3 POST /export_csv</u>	11
<u>6. Known Limitations</u>	12
<u>6.1 Contrastive Conjunctions Are Not Handled</u>	12
<u>6.2 Sarcasm Detection Is Pattern-Based, Not Semantic</u>	12
<u>6.3 Fixed, Non-Live Dataset</u>	12
<u>6.4 Keyword-Based Aspect Detection Has No Synonym Generalisation Beyond the Dictionary</u>	12
<u>6.5 Simulated, Not Real, Timestamps</u>	13
<u>7. Code Structure Appendix</u>	14
<u>7.1 File Overview</u>	14
<u>7.2 Key Functions Reference</u>	14

7.3 Dependency Summary

14

1. System Architecture

1.1 Architectural Overview

UniSentiment is a single-process, two-module Python application: a Flask web layer (`main.py`) and a standalone NLP engine (`sentiment_engine.py`) that the web layer imports as a library. There is no database, no separate frontend build step, and no external API dependency — the entire dashboard (HTML, CSS, and JavaScript) is embedded as a Python string and served from one Flask route, which keeps deployment to a single command: `python main.py`.

This monolithic-but-modular structure was a deliberate trade-off for an academic prototype: it removes deployment friction (no Node build, no Docker, no database migrations) while still keeping the NLP logic decoupled from the web logic, so the engine could be reused from a CLI script, a Jupyter notebook, or a different web framework without modification.

1.2 Module Responsibilities

Module	Responsibility
<code>main.py</code>	Flask app, HTML/CSS/JS dashboard template, REST routes (<code>/analyse</code> , <code>/export_csv</code>), request orchestration, CSV generation, browser auto-launch.
<code>sentiment_engine.py</code>	All NLP logic: tokenization, sentiment scoring, negation handling, sarcasm detection, aspect detection, aggregation, insight and alert generation.

1.3 Request Flow

A typical analysis request follows this path:

1. Browser loads the dashboard from GET `/` (served from the embedded HTML string in `main.py`).
2. User clicks "Run Analysis"; the dashboard's JavaScript POSTs a JSON payload of comments to `/analyse`.
3. Flask's `analyse()` route hands the comment list to the shared `_process()` helper in `main.py`.
4. `_process()` calls into `sentiment_engine.py` for each comment: `detect_aspects()`, `score_sentiment()`, and `score_aspect_sentiment()` for every detected aspect.
5. Per-comment results are aggregated into overall sentiment percentages, per-aspect summaries, fix-first / keep-doing / controversial insight lists, and threshold-based alerts.
6. The aggregated JSON is returned to the browser, where Chart.js renders the doughnut and bar charts and the dashboard populates metric cards, the comment explorer, and the alerts panel.

The `/export_csv` route mirrors this same `_process()` call, then serializes the per-comment records to an in-memory CSV (via `io.StringIO`) and streams it back as a file download, so analysis logic is never duplicated between the on-screen view and the export.

1.4 Frontend Stack

The dashboard uses Tabler (a Bootstrap-based admin UI kit) loaded from CDN for layout and components, Tabler Icons for iconography, and Chart.js for the two visualisations (a doughnut chart for

the overall positive/negative/neutral split and a bar chart for per-aspect performance). A gold-on-blue accent theme is layered on top of Tabler's default styling via custom CSS variables and utility classes (`.bg-gold`, `.text-gold`, `.btn-gold`) to match the branding used across all project deliverables.

Because the frontend is a single embedded string rather than a separate static-file tree, there is no build step (no webpack, no npm bundling) — editing the dashboard means editing the HTML string directly inside `main.py`.

1.5 Why No Database

The dataset is a fixed, hardcoded list of 60 realistic student comments (`RAW_COMMENTS` in `sentiment_engine.py`) rather than a live database table. This was an intentional scope decision for the assignment: it guarantees the same analysis results on every run (useful for grading and reproducibility), avoids introducing database setup as a dependency, and keeps the entire system runnable from a single Python file plus its pip dependencies. Section 6 (Known Limitations) discusses the consequences of this choice.

2. NLP Pipeline Deep Dive

2.1 Pipeline Stages

Every comment passes through the same five-stage pipeline inside `score_sentiment()`, regardless of whether it is being scored as a whole comment or as an aspect-isolated sentence group. The stages run strictly in order because each one depends on the output of the one before it:

1. NLTK tokenization — `word_tokenize()` splits the raw text into lowercase word tokens; `remove_stopwords()` produces a second, stopwords-filtered copy used for keyword/aspect evidence.
2. TextBlob base scoring — `textblob_polarity()` and `textblob_subjectivity()` return TextBlob's raw polarity (-1 to +1) and subjectivity (0 to 1) for the unmodified text.
3. Negation adjustment — `apply_negation()` inspects the token list for negation words and, if found, flips or dampens the polarity score.
4. Sarcasm override — `detect_sarcasm()` runs a set of regex patterns against the lowercased text; a match forces the polarity strongly negative, overriding everything computed in stages 2-3.
5. Label assignment — the final clamped polarity is mapped to positive / negative / neutral using fixed thresholds.

2.2 Negation Handling

Negation is detected by scanning the token list for any word in a `NEGATION_WORDS` set (e.g. "not", "cannot", "never"). When a negation word is found, the function looks ahead a fixed window of tokens for at least one meaningful content word (i.e. not a stopwords, and alphabetic) — this prevents a negation word at the very end of a sentence with nothing after it from triggering a false positive.

If negation context is confirmed, `apply_negation()` applies an asymmetric flip rather than a simple sign reversal:

```
def apply_negation(polarity, tokens):
    if detect_negation(tokens):
        return -polarity * (0.5 if polarity < 0 else 1.0)
    return polarity
```

The asymmetry is intentional: "not good" (TextBlob: +0.7) flips fully to -0.7, but "not bad" (TextBlob: -0.7) only flips to +0.35 — half the magnitude. This reflects a common pattern in sentiment linguistics where negated negative phrases ("not bad", "not terrible") read as mildly positive rather than as strongly positive as their un-negated antonym would be, and a full-magnitude flip would overstate that. The trade-off is a fixed 0.5 multiplier applied uniformly, rather than one tuned per word pair.

2.3 Sarcasm Detection

TextBlob has no built-in sarcasm awareness, since polarity scoring is lexical and irony is a pragmatic, context-dependent phenomenon. UniSentiment compensates with a layer of hand-written regex patterns (`SARCASM_PATTERNS`) matched against common ironic constructions student commenters actually used in the sample dataset — phrases such as "oh wow", "thanks university", and "very cool"

used in a complaining context (e.g. comment 9: "Oh wow, another parking ticket. Thanks university, very cool.").

When any pattern matches, the function does not simply flip the sign — it forces the score strongly negative regardless of what TextBlob or the negation stage computed:

```
if sarcasm_found:
    polarity = -abs(polarity) - 0.2
```

Taking the absolute value before negating guarantees the result is negative even if upstream stages had already produced a negative number (avoiding an accidental double-negative-to-positive flip), and the additional -0.2 offset pushes borderline cases clearly past the negative threshold rather than landing them in the neutral band.

2.4 Label Thresholds

The clamped polarity (forced into the -1.0 to +1.0 range) is converted to a label using a narrow neutral band around zero:

Polarity Range	Label
≥ 0.05	positive
≤ -0.05	negative
-0.05 to 0.05 (exclusive)	neutral

The ± 0.05 band (rather than a single zero cut-point) absorbs TextBlob's small amount of scoring noise on genuinely neutral statements, so factual or low-affect comments are not pushed into positive or negative purely by rounding.

2.5 Keyword Evidence Extraction

Alongside the numeric score, `score_sentiment()` also extracts human-readable supporting evidence: it runs TextBlob's POS tagger over the text, filters to adjectives (POS tags starting with "JJ"), and keeps those whose individual TextBlob polarity exceeds 0.1 (for positive evidence) or falls below -0.1 (for negative evidence), capped at five words each. This is what powers the "positive keywords" / "negative keywords" columns the dashboard and the exported dataset display alongside each comment — it gives a human reviewer a quick justification for why a comment was scored the way it was, without needing to re-read the full text.

3. Aspect Detection & Scoring

3.1 Aspect Dictionary

Aspects are not predicted by a model — they are detected deterministically via a keyword dictionary, `ASPECT_KEYWORDS`, mapping each of the system's eight tracked aspects (professors, wi-fi, parking, library, cafeteria, administration, campus safety, facilities) to a list of associated keywords and common synonyms/misspellings a student might use.

`detect_aspects()` lowercases the comment and tests each aspect's keyword list against it using word-boundary regex matching (`\b...\b`), so that, for example, the keyword "wifi" does not incorrectly match inside an unrelated longer word. The first keyword match for an aspect is sufficient — the function does not need every keyword to hit, only one. If no aspect's keywords match anywhere in the comment, the function falls back to a single ["general"] aspect so downstream code always receives a non-empty list to iterate over.

A single comment can, and frequently does, surface multiple aspects — e.g. comment 13 ("Enrolled expecting great things. Average professors, terrible food, Wi-Fi barely works.") triggers professors, cafeteria, and wi-fi simultaneously, and is scored independently against each one.

3.2 Sentence-Level Aspect Isolation

Scoring an aspect using the full comment's overall polarity would be inaccurate whenever a comment discusses multiple aspects with different sentiment (as in the example above, where professors and food/wifi pull in opposite directions). `score_aspect_sentiment()` solves this by first running NLTK's sentence tokenizer (`sent_tokenize`) on the comment, then keeping only the sentences whose text matches that aspect's keyword list.

If at least one sentence matches, only the joined relevant sentences are passed through the full `score_sentiment()` pipeline (stages 1-5 from Section 2) — meaning the score returned for, say, the "cafeteria" aspect of a multi-aspect comment reflects only the cafeteria-related sentence(s), not the whole comment. If no individual sentence happens to match (e.g. a single-sentence comment where the aspect keyword and the sentiment-bearing words are in the same clause), the function falls back to scoring the full comment text, so an aspect score is never left empty.

The function also returns the exact `relevant_text` string it scored, attached to the result dict — this is what makes the per-aspect scores auditable: a reviewer can see precisely which sentence(s) the aspect score is based on rather than trusting a black-box number.

3.3 Aspect Aggregation

`analyse_all_comments()` aggregates every individual aspect score across the full 60-comment dataset into per-aspect statistics: positive / negative / neutral percentage of mentions, an average compound (polarity) score, and a raw mention count. These aggregates are what feed both the dashboard's bar chart and the insight/alert logic described in Section 4.

4. Data Aggregation & Insights Engine

4.1 Insight Categories

Once per-aspect statistics are computed (Section 3.3), `_generate_insights()` (`sentiment_engine.py`) and the equivalent inline block in `main.py`'s `_process()` derive three actionable categories purely from fixed percentage thresholds — there is no machine learning involved at this stage, only rule-based filtering of the aggregated numbers:

Category	Rule	Purpose
Fix First	<code>negative_pct ≥ 40%</code> , sorted worst-first, top 3	Surfaces the aspects most urgently needing intervention.
Keep Doing	<code>positive_pct ≥ 60%</code> , top 3	Surfaces the university's clearest strengths, worth protecting/highlighting.
Controversial	<code>35% ≤ positive_pct ≤ 65%</code> AND <code>35% ≤ negative_pct ≤ 65%</code> , top 2-3	Surfaces polarising aspects with no clear consensus, worth qualitative investigation.

Aspects are first ranked by `avg_compound` score (most negative to most positive) before any threshold filtering, so that when more than three aspects qualify for a category, the most extreme (worst for Fix First, best for Keep Doing) are the ones surfaced rather than an arbitrary subset.

4.2 Alert Generation

Alerts are a separate, lower-level signal from insights: rather than ranking aspects, `_detect_alerts()` (`engine`) / the inline alert loop (`main.py`) fires one alert per aspect that crosses a negative-sentiment threshold, with two severity tiers:

Negative %	Severity	Example Message
<code>≥ 60%</code>	HIGH	"X% of parking mentions are negative — immediate action needed."
<code>40% - 59%</code>	MEDIUM	"X% of parking mentions are negative — monitor closely."
<code>< 40%</code>	(no alert)	—

The full pipeline version in `sentiment_engine.py` (`analyse_all_comments()`) additionally generates a trend-based alert: it walks the 14-day daily positive-sentiment trend day-over-day, and if the positive percentage drops by 20 points or more between any two consecutive days, it raises a HIGH severity "overall" alert flagging that specific date — this catches sudden sentiment swings that a static aspect-level threshold would miss.

4.3 14-Day Trend Simulation

Because the dataset is fixed rather than continuously collected, realistic-looking timestamps are generated by `generate_timestamps()`, which spreads the 60 comments pseudo-randomly across a 14-day window (with a fixed `random.seed(42)` so the same timestamps — and therefore the same trend

chart and trend-based alerts — are produced on every run). Each comment is bucketed into its day offset, and the daily positive percentage is computed per day, defaulting to 50% for any day with zero comments to avoid an undefined/zero-division result.

4.4 Keyword Frequency Analysis

Separately from sentiment scoring, `main.py`'s `_process()` builds a simple frequency table over the NLTK-tokenized, stopwords-filtered words across all processed comments (restricted to words longer than 3 characters to filter out residual noise tokens), and returns the top 20 most frequent as `top_keywords`. This is a descriptive-statistics feature rather than a sentiment feature — it gives a quick "what are people actually talking about" view independent of polarity.

5. API Reference

5.1 GET /

Returns the fully embedded dashboard (HTML, inline CSS, inline JavaScript) as a single response. No query parameters. This is the only route that serves a page rather than JSON.

5.2 POST /analyse

Runs the full analysis pipeline on a submitted comment list and returns the aggregated results as JSON.

Request Body

```
{
  "comments": ["comment text 1", "comment text 2", ...]
}
```

Response Body (200)

```
{
  "records": [ { "id", "comment", "aspects", "overall": {...} }, ... ],
  "total": <int>,
  "overall": { "positive_pct", "negative_pct", "neutral_pct", ... },
  "aspect_summary": { "<aspect>": { "positive_pct", "negative_pct", ... } },
  "fix_first": [...], "keep_doing": [...], "controversial": [...],
  "alerts": [ { "aspect", "severity", "message" }, ... ],
  "top_keywords": [ [word, count], ... ],
  "analysed_at": "<timestamp string>"
}
```

Returns 400 with {"error": "No valid comments"} if the comments array is missing or empty after filtering.

5.3 POST /export_csv

Accepts the same request body shape as /analyse, runs the identical `_process()` pipeline, then serialises the per-comment records into a CSV file streamed back as an attachment download (filename pattern `sentiment_YYYYMMDD_HHMMSS.csv`) rather than returning JSON.

CSV Columns

ID, Comment, Label, Polarity, Subjectivity, Aspects, Sarcasm, Negation, TextBlob Raw.

Because both routes call the same `_process()` helper, the on-screen dashboard and the exported CSV are always guaranteed to be consistent with each other — there is no separate export-specific recalculation that could drift out of sync with what the user sees on screen.

6. Known Limitations

6.1 Contrastive Conjunctions Are Not Handled

TextBlob scores polarity at roughly the whole-text/whole-sentence level using a bag-of-words-style lexicon approach; it has no syntactic awareness of contrastive conjunctions such as "but", "however", or "although", which in English typically signal that the clause following the conjunction should dominate the overall sentiment.

This produces a concrete, reproducible misclassification in the sample dataset: comment 20, "I expected to hate it here but professors have completely changed my perspective," is intended as positive (the "but" clause reverses an initially negative expectation), yet the system scores it negative, because the negative sentiment carried by "hate" in the first clause outweighs the milder positive language in the second clause once TextBlob and the negation layer have processed it. This was deliberately left uncorrected in the sample dataset and documented in the Project Report as well, rather than hand-tuned away, since it illustrates a genuine and common failure mode of lexicon-based sentiment scoring rather than a bug specific to this implementation.

A more complete fix would require either a dependency-parse-aware scorer that explicitly weights clauses after contrastive conjunctions more heavily, or a transformer-based sentiment model trained on contrastive examples — both of which are listed as future improvements in the Project Report.

6.2 Sarcasm Detection Is Pattern-Based, Not Semantic

The sarcasm detector relies entirely on a fixed list of regex patterns matching phrases the author anticipated ("oh wow", "thanks university", "very cool", etc.). This means: (a) sarcasm expressed through phrasing not in the pattern list will go undetected and be scored at face value, and (b) the same phrases used completely sincerely (e.g. an authentically enthusiastic "thanks university!") would be misclassified as sarcastic and forced negative. The patterns were tuned against the 60-comment sample dataset specifically, so detection accuracy on out-of-sample, real-world comments would likely be lower.

6.3 Fixed, Non-Live Dataset

RAW_COMMENTS is a hardcoded Python list of 60 comments rather than data pulled from a live source (e.g. a survey tool, social media, or a database of real student submissions). This guarantees reproducibility for grading purposes but means the system, as submitted, does not demonstrate ingestion of new or live data — a production deployment would need a data collection layer (form submission, API integration, or database) feeding into the existing scoring pipeline, which was intentionally left unbuilt to keep the assignment's scope manageable.

6.4 Keyword-Based Aspect Detection Has No Synonym Generalisation Beyond the Dictionary

Because aspect detection is exact keyword/regex matching rather than a semantic or embedding-based approach, any comment using vocabulary not present in ASPECT_KEYWORDS for a given aspect will not be attributed to that aspect, even if a human reader would clearly recognise the topic. Expanding the keyword lists is a low-effort mitigation, but a more robust solution (e.g. word-embedding similarity or a small classifier) is out of scope for the current implementation.

6.5 Simulated, Not Real, Timestamps

The 14-day trend and its day-over-day drop alert operate on timestamps generated by `generate_timestamps()` with a fixed random seed, not actual comment submission times. This was necessary because the underlying dataset has no real temporal metadata, but it means the trend chart and the trend-based alert are illustrative of the feature working correctly, rather than a finding about real sentiment-over-time behaviour.

7. Code Structure Appendix

7.1 File Overview

File	Lines (approx.)	Purpose
main.py	~1,440	Flask app, embedded dashboard (HTML/CSS/JS), REST routes, CSV export, browser launcher.
sentiment_engine.py	~530	NLP engine: dataset, tokenization helpers, scoring, negation, sarcasm, aspects, aggregation, insights, alerts.

7.2 Key Functions Reference

Function	Module	Role
<code>nlTK_tokenize()</code> / <code>remove_stopwords()</code>	<code>sentiment_engine.py</code>	Tokenization helpers used throughout the pipeline.
<code>textblob_polarity()</code> / <code>textblob_subjectivity()</code>	<code>sentiment_engine.py</code>	Thin wrappers around TextBlob's sentiment property.
<code>detect_negation()</code> / <code>apply_negation()</code>	<code>sentiment_engine.py</code>	Negation context detection and polarity adjustment (Section 2.2).
<code>detect_sarcasm()</code>	<code>sentiment_engine.py</code>	Regex-based sarcasm pattern matching (Section 2.3).
<code>score_sentiment()</code>	<code>sentiment_engine.py</code>	Master 5-stage scoring pipeline (Section 2.1).
<code>detect_aspects()</code>	<code>sentiment_engine.py</code>	Keyword-dictionary aspect extraction (Section 3.1).
<code>score_aspect_sentiment()</code>	<code>sentiment_engine.py</code>	Sentence-level aspect-isolated scoring (Section 3.2).
<code>generate_timestamps()</code>	<code>sentiment_engine.py</code>	Seeded pseudo-random timestamp simulation (Section 4.3).
<code>analyse_all_comments()</code>	<code>sentiment_engine.py</code>	Full pipeline entry point over the built-in dataset; used to generate the Project Report's results and the dataset deliverable.
<code>_detect_alerts()</code> / <code>_generate_insights()</code>	<code>sentiment_engine.py</code>	Threshold-based alert and insight derivation (Section 4.1-4.2).
<code>_process()</code>	<code>main.py</code>	Shared per-request pipeline used by both <code>/analyse</code> and <code>/export_csv</code> .
<code>analyse()</code> / <code>export_csv()</code>	<code>main.py</code>	Flask route handlers (Section 5).

7.3 Dependency Summary

- Flask — web server and routing.

- NLTK — tokenization (word/sentence), stopword corpus, POS tagging support corpora (punkt, punkt_tab, stopwords, wordnet, averaged_perceptron_tagger, averaged_perceptron_tagger_eng, brown, conll2000, movie_reviews).
- TextBlob — base polarity/subjectivity scoring and POS tagging used for keyword evidence extraction.
- Standard library — re (regex matching for aspects/sarcasm), random (seeded timestamp generation), datetime/timedelta, collections.defaultdict, io/csv (CSV export), threading/webbrowser (auto-launch).